

Déployer Airflow sur son propre EC2

Stack Airflow 3.x complétée (CeleryExecutor) sur une seule instance EC2, avec DAGs synchronisés depuis S3 afin de couvrir le déploiement, les pipelines d'ajout rencontrés en production, et les points opérationnels propres aux DAGs NappeCast.

- 1 Le secret
- 2 Rôle IAM EC2
- 3 Dockerfile
- 4 docker-compose
- 5 deploy.sh
- 6 Workflow CD
- 7 Après un reset DB
- 8 Pipelines rencontrés
- 9 Checklist

1 Secret airflow/prod-latest

CLÉ	RÔLE
AIRFLOW__CORE__FERNET_KEY	Chiffrement des connexions/variables sensibles en base
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN	Connexion Postgres de la metadata DB
AIRFLOW__CELERY__RESULT_BACKEND / BROKER_URL	Backend Celery (Postgres/Redis)
POSTGRES_PASSWORD	Mot de passe du conteneur Postgres local
_AIRFLOW_WWW_USER_USERNAME / _PASSWORD	Compte admin UI, créé une seule fois à l'init de la DB

CLÉ REDIS_PASSWORD MANQUANTE

Le service `redis` du `docker-compose` (étape 4) démarre avec `--requirepass "$REDIS_PASSWORD"`, mais `aws_create_secret.sh` ne définit **aucune clé REDIS_PASSWORD** dans le secret. En pratique la variable est vide, donc Redis démarre sans mot de passe (ou avec un mot de passe vide selon la version) ce qui fonctionne tant que rien d'autre ne s'y connecte, mais laisse le broker Celery non authentifié sur le réseau Docker interne. Ajoutez une clé `REDIS_PASSWORD` générée aléatoirement au secret, et vérifiez que `AIRFLOW__CELERY__BROKER_URL` l'inclut bien dans son URI (`redis://:PASSWORD@redis:6379/0`).

2 Rôle IAM de l'instance

Détail complet dans « Gestion des IAM ». Ce rôle couvre trois usages : lire le secret, tirer l'image ECR, et synchroniser DAGs/data/logs avec S3 via le service `airflow-s3-sync`.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": "arn:aws:secretsmanager:eu-west-3:844099234486:secret:airflow/prod-latest-*"
    }
  ],
}
```

```
{ "Effect": "Allow", "Action": "ecr:GetAuthorizationToken", "Resource": "*" },
{
  "Effect": "Allow",
  "Action": ["ecr:BatchGetImage", "ecr:GetDownloadUrlForLayer"],
  "Resource": "arn:aws:ecr:eu-west-3:844099234486:repository/airflow"
},
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket", "s3:DeleteObject"],
  "Resource": ["arn:aws:s3:::nappecast", "arn:aws:s3:::nappecast/airflow/*"]
}
]
```

3 Dockerfile

```
DOCKERFILE Ã€” DOCKER/AIRFLOW/DOCKERFILE
FROM apache/airflow:3.1.3

USER root
RUN apt-get update && apt-get install -y --no-install-recommends \
  gosu \
  libpq-dev \
  && apt-get clean && rm -rf /var/lib/apt/lists/*
USER airflow

COPY requirements.txt .

RUN PYTHON_VERSION=$(python3 -c 'import sys; print(f"{sys.version_info[0]}.{sys.version_info[1]}")') && \
  pip install --no-cache-dir \
    "apache-airflow==${AIRFLOW_VERSION}" \
    -r requirements.txt \
    --constraint "https://raw.githubusercontent.com/apache/airflow/constraints-${AIRFLOW_VERSION}/constraints-${PYTHON_VERSION}.txt"

ENV PYTHONPATH=/opt/airflow
```

TEXT Ã€” REQUIREMENTS.TXT

```
apache-airflow-providers-postgres
apache-airflow-providers-amazon
pandas
requests
pyyaml
python-dotenv
```

POURQUOI AUCUNE VERSION N'EST Ã€PINGLÃ€E

Toute version fixÃ€e ici (ex. `pandas==2.3.3`) qui diverge de celle pinÃ€e par le fichier `--constraint` officiel d'Airflow provoque un `pip.ResolutionImpossible` au build. Laisser pip choisir dans les bornes du fichier de contraintes est la faÃ§on la plus fiable d'Ã©viter ce conflit Ã€ quitte Ã€ perdre un peu de reproductibilitÃ© stricte entre deux builds.

IMPORT MORT DANS UN DAG Ã€ CASSE POTENTIELLE DU PARSING

`dags/nappecast_daily_predict.py` contient `from src.api.main import forecast`, jamais utilisÃ© dans le fichier. Le module `src/` (code de l'API FastAPI) n'est ni copiÃ© dans l'image

Airflow, ni synchronisé depuis S3 par `airflow-s3-sync` → cet import échouera dès que Python essaiera de parser ce DAG dans ce conteneur, avec un DAG qui disparaît silencieusement de l'UI (erreur visible seulement dans les logs du `dag-processor`). Si le déploiement actuel semble fonctionner, c'est probablement parce que le fichier n'a pas encore été re-synchronisé depuis un commit propre, ou parce qu'une dépendance transitive rend l'import muet → **À valider et supprimer cette ligne** avant le prochain déploiement.

4 docker-compose.yml

Services Airflow 3.x : `postgres`, `redis`, `airflow-apiserver` (remplace `webserver` en 3.x), `airflow-scheduler`, `airflow-dag-processor` (service séparé en 3.x), `airflow-worker`, `airflow-triggerer`, `airflow-init`, `flower`, et `airflow-s3-sync`.

YAML → EXTRAIT CLÉ, SERVICE DE SYNCHRONISATION S3

```
airflow-s3-sync:
  image: amazon/aws-cli:2.17.62
  env_file:
    - airflow.env
    - .env.runtime
  volumes:
    - ./dags:/dags
    - ./data:/data
    - ./logs:/logs
  entrypoint: ["sh", "-c"]
  command:
    - |
      while true; do
        aws s3 sync "s3://$S3_BUCKET/${DAGS_S3_PREFIX:-dags}" /dags --delete
        aws s3 sync "s3://$S3_BUCKET/${DATA_S3_PREFIX:-data}" /data
        aws s3 sync /logs "s3://$S3_BUCKET/${LOGS_S3_PREFIX:-logs}"
        sleep "${SYNC_INTERVAL_SECONDS:-60}"
      done
  restart: unless-stopped
```

POURQUOI \$\$VAR ET PAS \${VAR} ICI

L'interpolation `${VAR}` de Compose ne lit que depuis un fichier littéralement nommé `.env` → c'est tout du compose → jamais depuis les fichiers listés en `env_file`. → écrire `$$VAR` dans le `command` → échappe le signe pour Compose et laisse le `sh` du conteneur la résoudre → l'exception, → partir des variables que `env_file` lui a bien injectées. C'est la correction appliquée après l'erreur `S3_BUCKET required variable is missing a value` rencontrée en déploiement réel (détail → tape 8).

Bind mounts (pas de volumes nommés) pour `./dags`, `./data`, `./logs` → cohérent avec une synchronisation S3 qui doit voir les mêmes fichiers que les autres services Airflow.

5 deploy.sh

BASH → EXTRAIT

```
mkdir -p dags logs plugins config data
chown -R 50000:0 dags logs plugins config data

docker compose --env-file airflow.env pull
docker compose --env-file airflow.env up -d --remove-orphans
```

`--env-file airflow.env` est passé explicitement à chaque appel `docker compose` afin nécessaire pour les valeurs encore résolues par Compose lui-même (ex. `user:`, chemins de volumes), puisque le fichier ne s'appelle pas `.env`.

6 Workflow CD (`cd-airflow.yml`)

SECRET GITHUB

USAGE

<code>AIRFLOW_SG_ID</code>	Security group dont le port 22 est ouvert/fermé le temps du déploiement
<code>AIRFLOW_EC2_HOST</code> / <code>_USER</code> / <code>_SSH_KEY</code>	Connexion SCP/SSH à cette instance

`command_timeout: 30m` sur l'attente SSH afin nécessaire, le premier `pull` de l'image Airflow (assez lourde) après un redémarrage à froid pouvant dépasser le timeout par défaut de 10 min de `appleboy/ssh-action`.

7 À refaire après tout reset de la metadata DB

Un `docker compose down -v` (utilisé par exemple pour résoudre une incompatibilité de version Postgres, voir à tape 8) efface la base de métadonnées Airflow afin donc tout ce qui n'est pas dans le code des DAGs ou dans le secret. Trois éléments à recréer manuellement :

- ✓ **Variable `pipeline_secret`** afin utilisé par les trois DAGs sans valeur par défaut (`Variable.get("pipeline_secret")`), doit correspondre au `PIPELINE_SECRET` du secret `app_api` : `airflow variables set pipeline_secret "<valeur>"`.
- ✓ **Variable `nappecast_api_base_url`** afin a un défaut (`http://172.31.41.22:8000`), donc moins critique, mais à vérifier si l'IP privée de l'instance `app_api` a changé depuis.
- ✓ **Pool `training_pool`** afin requis par `nappecast_monthly_retrain.py` (`pool="training_pool"`) ; sans lui la tâche reste en `queued` indéfiniment. À créer via Admin à Pools (1 slot suffit pour un entraînement séquentiel) ou `airflow pools set training_pool 1 "Entraînement mensuel"`.

8 Problèmes rencontrés en déploiement réel

SYMPTÔME

CAUSE

CORRECTIF

<code>airflow: command not found</code>	Base image <code>2.11.0</code> + <code>pip install apache-airflow==3.1.3</code> par-dessus	Aligner <code>FROM</code> sur <code>\${AIRFLOW_VERSION}</code>
<code>PYTHON_MAJOR_MINOR_VERSION: unbound variable</code>	Variable inexistante dans cette image de base	Calculer via <code>python3 -c 'import sys; ...'</code>
<code>pip.ResolutionImpossible</code>	Version à pingler en conflit avec le	

(pandas, requests)	fichier de contraintes	Retirer les pins de version
airflow-postgres-1 unhealthy " database files are incompatible	Volume avec donn�es Postgres 13 sous image Postgres 16	docker compose down -v (perte de la metadata DB, voir �tape 7)
S3 BUCKET required variable is missing a value	\${VAR} dans le YAML r�solu uniquement depuis .env , jamais depuis env_file	��chapper en \$\$VAR dans command: , --env-file airflow.env explicite
ECR : tag immutable	Image d�j� pouss�e sur ce SHA lors d'un run pr�c�dent	Supprimer l'image via ecr batch-delete-image ou relancer les jobs �chou�s plut�t qu'un nouveau run
Rebuild inutile malgr� tag existant	R��le CI sans ecr:DescribelImages �t' l' AccessDenied ��tait interpr�t�� comme �� image absente ��	Ajouter ecr:DescribelImages � la policy du r��le CI
no space left on device	Volume racine 8 Go trop petit pour l'image + d�pendances	Agrandir le volume EBS (8�25 Go) : modify-volume , growpart , xfs_growfs
Timeout SSH pendant le d�ploiement	Timeout par d�faut (10 min) de appleboy/ssh-action trop court pour un premier pull	command_timeout: 30m

9 Checklist

   CI   REDIS_PASSWORD ajout   au secret et r  f  renc  e par AIRFLOW_CELERY_BROKER_URL .

   Import mort from src.api.main import forecast supprim   de nappecast_daily_predict.py .

   Variables pipeline_secret / nappecast_api_base_url et pool training_pool recr   s apr  s tout reset de la DB.

   R  le IAM d'instance en place pour Secrets Manager, ECR et S3.

   Volume EBS surveill      25 Go actuellement,    r    valuer si de nouvelles d pendances sont ajout  es.

Voir    Gestion des IAM    pour les r  les,    Pipeline Secrets AWS    pour le m  canisme g  n  rique de secret,    D ploiement app_api    pour le reverse proxy qui expose airflow.mapflowing.com .